

## 树上问题 1

我听说，从前有个人，闲来无事，想找个人一起下棋。他站在院子里，喊了一声：“谁有空与我手谈一局？”结果邻居们各有各的事，有的要浇园，有的要喂鸡，有的在补衣裳。他便垂着手站在门口，等了一会儿，又喊了一遍。这时有个路过的人说：“你喊了也没用，大家都有自己的本分。”那人听了，想了想，觉得也对，便自己回屋摆开棋盘，左手与右手对弈，倒也下出了一番道理。如今我喊一声“谁有空跟我打游戏”，其实也是一样的道理。游戏这东西，本是人与人之间的交际，讲的是你来我往的礼数。若无人应答，我便该明白，此刻大家各有各的职分，各有各的忙处。我不该强求，也不必失落。自己开一局，练练手，等有人得空了，自然有人应声。这就可以说是：有伴便同乐，无伴便独行，二者都不失去体面。

Cap1taL

# 树上背包 DP 的复杂度

- 树上背包转移时枚举子树大小，需要记录前缀子树的子树和，而非真正的  $sz_u$
- 复杂度证明考虑所有的点对
- 在容量与  $n$  同阶时为  $O(n^2)$ ，在  $m \ll n$  时为  $O(nm)$

# 树链剖分

- 树链剖分可以把一棵树划分成若干链（自上而下的链），以便于维护树上信息
- 树链剖分一般会对链内重标号，并保证一条链内点的编号连续。
- 因此，我们可以用维护序列上的来维护一条链上的信息
- 对于跨链的信息（例如路径），我们一般将其拆分成若干链。合适的树链剖分方式往往可以保证链的数量较少，从而保持较优的复杂度

# 重链剖分

- Code <https://www.luogu.com.cn/paste/vqjegxoq>
- 选择最重的儿子进行链剖分延伸
- 链切换次数为  $O(\log n)$ ，证明考虑每次换链一定子树大小翻倍
- 链上编号连续，子树编号也连续
- 可以把一个路径拆为  $O(\log n)$  个序列上的区间，或者维护子树信息
- 常数很小

# 长链剖分

- Code <https://www.luogu.com.cn/paste/vqjegxoq>
- 选择最深的儿子进行链剖分延伸
- 链切换次数为  $O(\sqrt{n})$ ，证明考虑每次换链一定多一条更长的链
- 由于链切换次数较劣，我们一般不用长链剖分维护路径或子树等树上信息

# 长链剖分优化 DP

- 类似树上启发式合并的一种 trick
- 可以把与深度有关的信息维护加速到线性
- 暴力做这类信息维护一般是  $O(n^2)$  的
- 具体来说，每次直接继承长儿子的信息，暴力合并轻儿子的信息
- 继承是快速的，一个链只有在链顶向上合并的时候才需要完整遍历，总复杂度  $O(n)$
- 由于不能完整开数组，实现一般采用指针 + 动态分配内存，或者使用 vector 与 swap。

# 长链剖分求 $k$ 级祖先

- 注意到关键性质：一个点  $k$  级祖先所在的长链长度必  $\geq k$
- 反证法，若小于  $k$ ，则选取当前这个链更优，这个才是长链
- 因此若我们先跳一次  $2^h$  祖先，剩余高度不会超出所在链的预处理范围
- 对于每个链顶，设链长为  $L$ ，处理其上下各  $L$  个点，跳一次  $2^h$  后直接处理即可
- 预处理  $O(n \log n)$ ，查询  $O(1)$
- 有时仍然不如重链剖分快

# 树上倍增求 LCA

- 预处理一个点的  $2^k$  级祖先
- 求 LCA 的时候先跳平，再从大到小尝试跳到 LCA 下方
- 预处理  $O(n \log n)$ ，查询  $O(\log n)$ ，空间  $O(n \log n)$ ，实际运行超级慢



# 重链剖分求 LCA

- 预处理重链剖分，不需要重标号，只需要处理链顶
- 不断跳重链，直到跳到同一个链上，较浅的一个即为 LCA
- 预处理  $O(n)$ ，查询  $O(\log n)$ ，空间  $O(n)$ ，实际运行超级快

# DFN 序求 LCA

- 考虑  $u, v$  不是祖先后代的情况。显然 dfn 序列上  $u, v$  之间的元素都在  $lca$  的子树中。只需要找到深度最小的元素，其父亲即为 LCA。这是一个 RMQ 问题，可以使用 ST 表。
- 对于他们是祖先后代的情况，只需要去掉  $u$ ，即可兼容上一种情况。
- 具体实现并不需要维护父亲与深度，只需要在 ST 表底层直接填写父亲，找 dfn 序最小的即可。容易理解这个技巧的正确性
- Code: <https://www.luogu.com.cn/record/152919187>
- 预处理  $O(n \log n)$ ，查询  $O(1)$ ，空间  $O(n \log n)$ ，常数远小于树上倍增欧拉环游序

# 点分治

- 又称重心分解，可以用来处理大规模的路径问题等
- 我们在树上选择一个根，则路径可以按照是否经过与经过的情况，被分为三种
- 若我们能适当地选取根，则可以高效地批量分类并处理这些路径
- 每次树根，暴力遍历整个极大联通块内的各个子树，维护信息的同时求出后续的新重心，复杂度是  $O(n \log n)$  的

# 点分树

- 把点分治的形态固定成一棵树，属于一种重构树
- 可以处理一类与树形态无关的问题
- 关键问题在于，如何去除点分树上子树对父节点的影响
- 常见做法是对于每个节点维护两个数据结构，代表字数内点与自己或与父亲的信息
- 另一种做法是维护前缀与后缀数据结构，常见于修改极其特殊的情形

# P4315 月下“毛景树”

给定一棵有  $N$  个节点、 $N - 1$  条树枝的树，毛毛果长在树枝上。支持以下操作：

Change  $k\ w$  : 将第  $k$  条树枝的毛毛果个数改为  $w$

Cover  $u\ v\ w$  : 将  $u$  到  $v$  路径上所有树枝的毛毛果个数改为  $w$

Add  $u\ v\ w$  : 将  $u$  到  $v$  路径上所有树枝的毛毛果个数增加  $w$

Max  $u\ v$  : 查询  $u$  到  $v$  路径上树枝毛毛果个数的最大值

操作以 Stop 结束。

$1 \leq N \leq 10^5$ ，操作和询问数目不超过  $10^5$ 。

# Solution

- 树剖模板
- Code: <https://www.luogu.com.cn/record/109990824>

# CF1009F Dominant Indices

给定一棵以 1 为根的树，包含  $n$  个顶点。对于每个顶点  $x$ ，定义  $d_{x,i}$  为：在  $x$  的子树中，与  $x$  距离恰好为  $i$  的顶点数。顶点  $x$  的主导下标  $j$  是满足以下条件的最小下标：

$$d_{x,j} = \max_{k \geq 0} d_{x,k}.$$

换句话说， $j$  是  $d_{x,j}$  第一次取到最大值的深度。请计算每个顶点的主导下标。

$1 \leq n \leq 10^6$ 。

# Solution

- 可以写出一个暴力 DP。设  $f_{u,i}$  表示  $u$  这个点下面深度为  $i$  的点数

$$f_{u,i} = \sum_{u \rightarrow v} f_{v,i-1}$$

- 暴力做显然是  $n^2$  的
- 采用长链剖分优化。对于每个点：
  - 直接继承长儿子的 DP 数组
  - 在数组最前面添加一个 1 代表  $f_{u,0}$
  - 暴力合并所有短儿子
  - 复杂度  $O(n)$



# P3806 【模板】点分治

给定一棵有  $n$  个点的树，边有边权。共有  $m$  次询问，每次询问一个整数  $k$ ，判断树上是否存在一对点，使得它们之间的距离恰好为  $k$ 。

$1 \leq n \leq 10^4$ ,  $1 \leq m \leq 100$ ,  $1 \leq k \leq 10^7$ ,  $1 \leq w \leq 10^4$ 。

# Solution

- 点分治
- 每次分治时遍历子树，记录到根的距离，等价于拆点
- 使用某些数据结构或者桶支持查询即可
- 某种意义上类似树上 DP 的转移
- 询问数较少，可以暴力处理。复杂度  $O(nm \log n)$
- Code: <https://www.luogu.com.cn/record/291033843>

## P6329 【模板】点分树 / 震波

给定一棵  $n$  个点的树，相邻两个城市距离为 1，第  $i$  个城市价值为  $value_i$ 。在线处理  $m$  次操作：

0 x k: 查询与  $x$  距离不超过  $k$  的所有城市的价值和。

1 x y: 将第  $x$  个城市价值改为  $y$ 。

强制在线

$1 \leq n, m \leq 10^5$ ,  $1 \leq value_i, y \leq 10^4$ ,  $0 \leq k \leq n - 1$ 。

# Solution

- 建立点分树
- 每个点维护两个数据结构，存储点分树子树内与自己点分树父亲距离  $\leq k$  的价值信息
- 查询时暴力在点分树上上跳，每次计算都扣除子树影响
- 修改同样暴力跳点分树，维护两个数据结构
- 此题的实现可以使用树状数组，时间复杂度  $O(n \log^2 n)$ ，空间复杂度  $O(n \log n)$

## P2056 [ZJOI2007] 捉迷藏

给定一棵由  $N$  个房间和  $N - 1$  条双向走廊组成的树，初始所有房间的灯都是关闭的。孩子们只会躲藏在没有开灯的房间中。每次操作为：

C  $i$  : 改变第  $i$  个房间的开关状态（开  $\leftrightarrow$  关）

G : 查询最远的两个关灯房间的距离

对于每个 G 操作：若只有一间关灯房间，输出 0；若没有关灯房间，输出 -1。

$1 \leq N \leq 100000$ ,  $1 \leq Q \leq 500000$ 。

# Solution

- 建立点分树
- 每个点维护两个数据结构，存储点分树子树内与自己 点分树父亲的距离信息
- 查询时暴力在点分树上上跳，每次计算都扣除子树影响
- 修改同样暴力跳点分树
- 具体实现可以维护两类可删堆，一类是存储所有路径的堆，一类是存储各个子树最大值的堆。或者使用线段树等。总之要维护子树到根路径信息
- 时间复杂度  $O(n \log^2 n)$ ，空间复杂度  $O(n \log n)$

## P9058 [Ynoi2004] rpmtdq

给定一棵有边权的无根树，共有  $n$  个节点。定义  $\text{dist}(i, j)$  表示树上点  $i$  和点  $j$  之间的距离。

有  $q$  次询问，每次给出  $l, r$ ，要求出：

$$\min_{l \leq i < j \leq r} \text{dist}(i, j)$$

若不存在满足  $l \leq i < j \leq r$  的二元组  $(i, j)$ ，则输出  $-1$ 。

$1 \leq n \leq 2 \times 10^5$ ,  $1 \leq q \leq 10^6$ ,  $1 \leq z \leq 10^9$ 。

# Solution

- 寻找支配点对
- 固定分治根，考虑什么样的点对无效，即对于  $i < j < k$ ，若  $dis_{rt,k} > dis_{rt,j}$ ，则  $k$  不可能成为  $i$  的答案
- 更具体地，若  $\max(dis_i, dis_j) < \min_{i < k < j} dis_k$ ，则  $i, j$  是有效点对
- 考虑如何找到这些点对
- 我们按照编号升序扫描分治块内的点，维护不降单调栈，弹栈时完成支配点对的配对。个数是  $O(n \log n)$
- 然后转化为一个二维排序的问题，类似于查询二维区域内最小值，随便解决即可



*Thanks!*